CPSC-354 Report

Brandon Foley
Chapman University

September 11, 2025

Abstract

Contents

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 Homework

Here is my work on the MU Puzzle:

$$MIU \to MIUIU \to MIUIUIU \to \cdots$$

$$MI \to MII \to MIIII \to MUI \quad \Rightarrow \quad MUI$$

$$MUI \to MUIU$$

$$MUI \to MUIU \to MUIUIU \to MUIUIUIU \to MUUIU \to MUUIUIU \to \cdots$$

$$MUIIU \to MUIIUUIU \to MUIU$$

$$MUIIU \to MUUIU \to MUIU$$

$$MUIU \to MUUIU \to MUIUIU \to MUIUIUIU \to \cdots$$

$$MUIUIUIU \to MUIUIUIUIU \to MUIUIUIUIUIU \to \cdots$$

The MU Puzzle is unsolvable because it is impossible to reach the string `MU` starting from `MI` using the given rules. After a little research and thinking on my part it is the number of `I`'s that make this impossible: rule applications can double the count, add one, or remove three at a time, however, none of these operations ever reduce the number of `I`'s to exactly zero. Since `MU` has no `I`'s, it can never be derived.

2.1.2 Questions

What rules could we add or remove in order to make this possible? I was thinking are there any ways to make this question solvable by adding only one more rule or maybe altering a rule. I think that this is an interesting question and concept.

2.2 Week 2

2.2.1 Homework

Here is my work on the Rewriting Homework:

Rewriting

1. $A = \{\}$

2. $A = \{a\}$ $R = \{\}$

$$a$$

3. $A = \{a\}$ $R = \{(a,a)\}$

$$\begin{tikzcd} a \arrow[loop right] \end{tikzcd}$$

4. $A = \{a,b,c\}$ $R = \{(a,b),(a,c)\}$

$$\begin{tikzcd} a \arrow[r] \arrow[dr] & b \\ & c \end{tikzcd}$$

5. $A = \{a,b\}$ $R = \{(a,a),(a,b)\}$

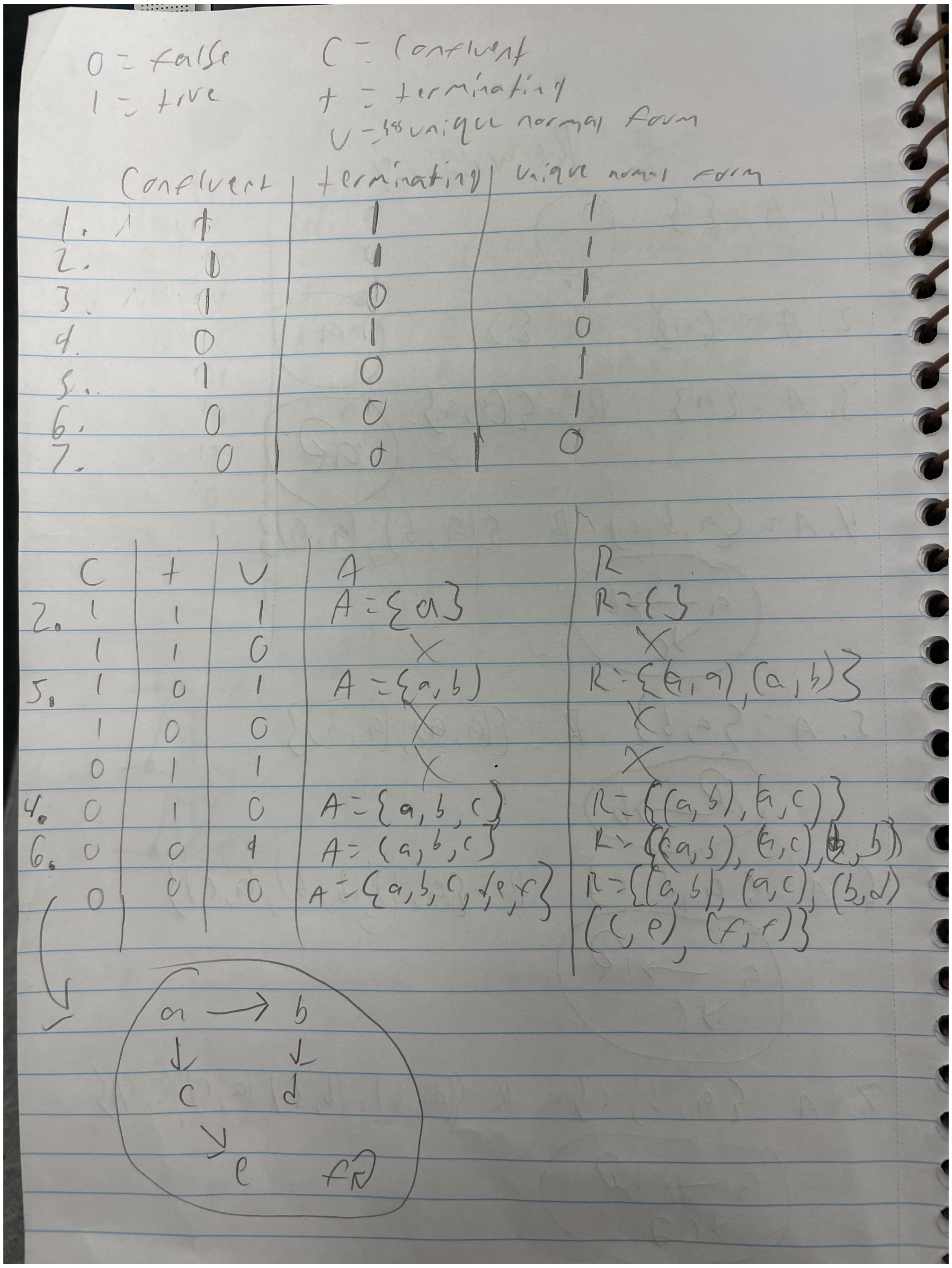
$$\begin{tikzcd} a \arrow[loop above] \arrow[r] & b \end{tikzcd}$$

6. $A = \{a,b,c\}$ $R = \{(a,b),(b,b),(a,c)\}$

$$\begin{tikzcd} a \arrow[r] \arrow[dr] & b \arrow[loop right] \\ & c \end{tikzcd}$$

7. $A = \{a,b,c\}$ $R = \{(a,b),(b,b),(a,c),(c,c)\}$

$$\begin{tikzcd} a \arrow[r] \arrow[dr] & b \arrow[loop right] \\ & c \arrow[loop right] \end{tikzcd}$$



In our homework we discovered 3 seperate cominations that are impossible to create. The first confluent, terminating and no unique normal form is impossible because if a system is confluent and terminating then it must have a unique normal form. The second confluent, not terminating and no unique normal form is impossible because if a system is confluent but not terminating then it must have a unique normal form when it exists. The third not confluent, terminating and unique normal form is impossible because if a system is terminating and has a unique normal form then it must be confluent.

2.2.2 Questions

When relating to programming, if a system is confluent but not terminating, like in some programs with infinite loops, how does confluence still guarantee that the result of a program is unique when it exists?

2.3 Week 3

2.3.1 Homework

Here is the work on ARS rewriting HW:

Exercise 5:

- Reduce some example strings such as `abba` and `bababa`.

$$abba \to baba \to baab \to bb \to b \to \{\}$$

$$bababa \to baabba \to bbba \to bba \to ba \to a$$

- Why is the ARS not terminating?
  The ARS is not terminating because you can infinitely go from ab to ba and vice versa.
- How many equivalence classes does $\leftrightarrow^*$ have? Can you describe them in a nice way? What are the normal forms?
  There are two equivalence classes: words with an even number of a's vs words with an odd number of a's.
  The normal forms are the empty string $\{\}$ and the string `a`.
- Can you change the rules so that the ARS becomes terminating without changing its equivalence classes?
  You can change $ba \to ab$ to $ba \to b$. This keeps the same equivalence classes but makes the system terminating.
- Write down a question or two about strings that can be answered using the ARS.
  - Does this string reduce to the empty string?
  - Are two given strings equivalent under the ARS?

Exercise 5b:

- Reduce some example strings such as `abba` and `bababa`.

$$abba \to baba \to baab \to bab \to ba \to a$$

$$bababa \to baabba \to babba \to baba \to aba \to aa \to a$$

- Why is the ARS not terminating?
  The ARS is not terminating because you can infinitely go from ab to ba and vice versa.
- How many equivalence classes does $\leftrightarrow^*$ have? Can you describe them in a nice way? What are the normal forms?
  There is one equivalence class: all words reduce to one a.
  The normal form is just `a`.

- Can you change the rules so that the ARS becomes terminating without changing its equivalence classes?
  You can change $ba \to ab$ to $ba \to b$. This keeps the same equivalence classes but makes the system terminating.
- Write down a question or two about strings that can be answered using the ARS.
  - Is there a unique normal form for this ARS?
  - Are two given strings equivalent under the ARS?

Bonus: Exercise 8

- Starting configuration:

$$
\begin{aligned}
&aaaaaaaaaaaaaaabbbbbbbbbbbbbbccccccccccccc \\
&\to^* acccccccccccccccccccccccccccccccccccccccc \\
&\to bbcccccccccccccccccccccccccccccccccccccccc \\
&\to^* bbbbbbbbcccccccccccccccccccccccccccccccccc
\end{aligned}
$$

- It is impossible to reach a configuration of only a's, only b's, or only c's.
- At each step, the rules correspond to the following change-vectors:

$$(a-1,\, b-1,\, c+2), \quad (a-1,\, b+2,\, c-1), \quad (a+2,\, b-1,\, c-1)$$

- Reducing these modulo 3 gives

$$(-1,\, -1,\, -1)$$

  for each move (since $+2 \equiv -1 \pmod 3$).
- Thus, we can simply subtract the numbers of a's, b's, and c's directly:

$$a - b = 15 - 14 = 1 \pmod 3$$

$$b - c = 14 - 13 = 1 \pmod 3$$

- Since these differences never reduce to 0, there will always be one leftover letter that prevents the string from reducing to a uniform string of only a's, b's, or c's.

2.4 Questions

Given the rules for changing letters in exercise 8 ARS, why is it impossible to end up with a string containing only one type of letter, and how can we use modular arithmetic to prove it?

3 Essay

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.